



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Aprendizaje Automático

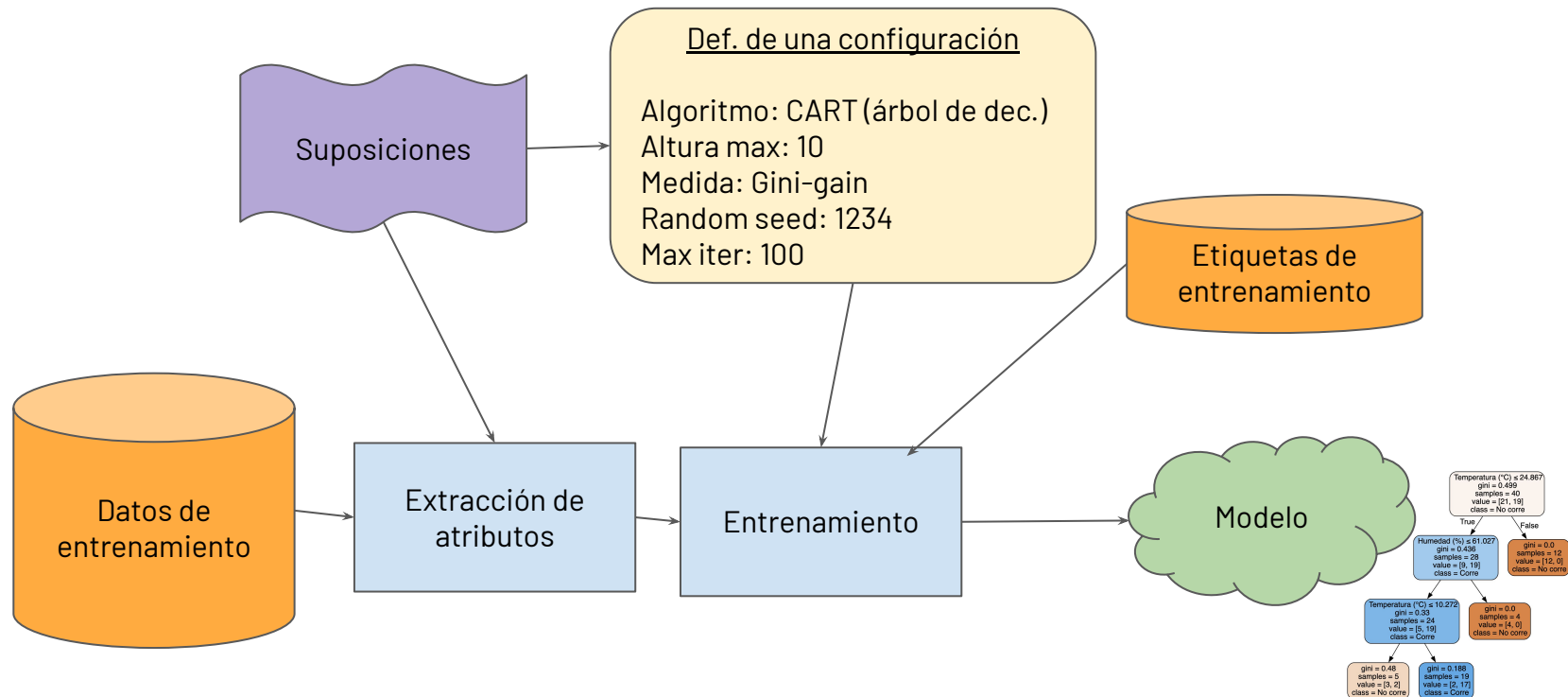
Clase 3:

Evaluación y Selección de Modelos

¿Qué debería saber después de esta clase?

- Conocer estrategias seguras de **evaluación**
- Diferencia entre **sobreestimación** y **sobreajuste**.
- Diferencia entre Model **Selection** y Model **Assessment**.
- Metodologías seguras de **separación de datos**. **Desarrollo**, **Control**.
- Técnicas **grid search**, **randomized search**, etc.
- Técnicas **validación cruzada** (simple, k-fold, group k-fold).

Repaso pipeline de entrenamiento



Etapas de evaluación: Importancia

Casi todo lo que hacemos en Aprendizaje Automático depende en gran medida de la evaluación:

- **Durante el desarrollo: comparamos** configuraciones de algoritmos mediante mediciones con alguna métrica.
- **Al finalizar el desarrollo: estimamos** la performance “en la realidad” (in the wild) y la reportamos.
- ★ Es **fácil errar** en la creación de un modelo (entrenamiento), pero también es **fácil darse cuenta**.
- ★ Es **fácil errar** en la evaluación y **no es fácil darse cuenta**.
- No es raro ver un papers o sistemas en producción con problemas en la evaluación que no funciona.

Evaluar **bien** significa entender bien el **caso de uso**,

- entender **qué me importa del problema** y qué no,
- entender **qué métrica/s** refleja/n lo que quiero capturar,
- entender **qué mecanismo de evaluación** usar,
- entender **cómo no hacer/se trampa**
- entender **cómo hago que no me hagan trampa**.

¿Qué puede **salir mal**? (algunos ejemplos)

- Errores **en los datos** (en sí o en su uso).
- **Evaluar** en datos que usamos para **entrenar** (obvio).
- **Evaluar** en datos que usamos para **desarrollar** (no tan obvio).
- Poca (o nula) **significancia estadística**
- **Datos no representativos** de la “realidad”.
- Perder de vista que existen **grupos en los datos**.
- Fallar en la elección de **métricas** (clase 5)

Ejemplo: Predicción de embarazos en Salta

Cómo hacer las cosas **mal**

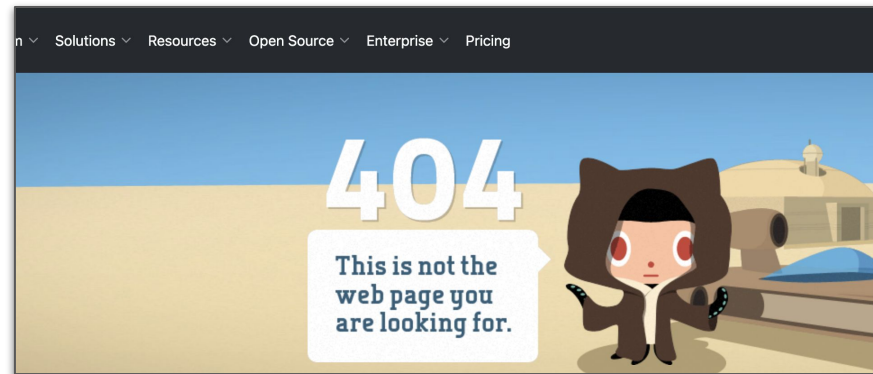
<https://liaa.dc.uba.ar/es/sobre-la-prediccion-automatica-de-embarazos-adolescentes/> (autobombo alert)

Sobre la predicción automática de embarazos adolescentes

Tras estudiar la metodología del sistema de inteligencia artificial supuestamente capaz de predecir embarazos adolescentes, mencionado por el Gobernador de Salta, Juan Manuel Urtubey, encontramos serios errores técnicos y conceptuales, que ponen en duda los resultados reportados y comprometen el empleo de dicha herramienta, sobre todo tratándose de una cuestión tan sensible.

El proceso utilizado para obtener los resultados reportados es técnicamente incorrecto. Se está violando un principio básico del aprendizaje de máquinas: que los datos sobre los que se evalúa el sistema deben ser distintos a los datos que se usan para entrenarlo. Si este principio se viola, es decir, si hay contaminación de datos de entrenamiento en los datos sobre los cuales se valida, los resultados serán inválidos.

En el sistema descrito en github por el autor, la contaminación de los datos de evaluación surge de manera bastante sutil. El sistema usa un método para balancear la cantidad de muestras de cada clase llamado SMOTE. Este método genera nuevas muestras "sintéticas" replicando las



<https://github.com/facundod/case-studies/blob/master/Prediccion-de-Embarazo-Adolescente-con-Machine-Learning.md>

Otros reportes del caso:

- [No culpen al algoritmo: ¿cómo regular la inteligencia artificial en un mundo demasiado humano? · Fundar](#)
- [El Caso del Inquietante Algoritmo Que 'Predice' el Embarazo Adolescente | WIRED](#)

Overview del pipeline completo

Cómo hacer las cosas **bien**

- 1) **Separar** datos para **control** (slide que sigue).
- 2) **Definición** del problema (clase 1) y métricas (clase 5)
- 3) Etapa de **Desarrollo**:
 - a) Análisis exploratorio de datos y metadata.
 - b) Definir configuraciones iniciales a evaluar.
 - c) Selección del mejor modelo (en un par de slides)
 - i) Evaluar cada configuración de manera segura.
 - d) Análisis de errores.
 - e) Repetir desde 2.a) si es necesario.
 - f) Entrenamiento del modelo final.
- 4) **Estimación** de rendimiento en la realidad (slide que sigue).
 - a) Construir reportes finales.

Primer paso: Conjunto de Control

Antes de **hacer cualquier otra cosa** separamos un conjunto de datos que guardaremos en una caja fuerte

(datos_control.zip pass=<cumple_de_pablo>).

- **No se usan hasta el final del desarrollo:**
 - Hasta unos días antes de publicar el paper
 - Hasta unos días antes de subir el sistema a producción
- **¡Se usan una vez! (solo para reportar!)**
 - Para evaluar el modelo que se sube a prod.
 - O para armar la tabla final del paper
 - **No para tomar decisiones. No para elegir los mejores.**
- **“No me gusta lo que me dio”:**
 - Ajo y agua. Reportarlo igual que seguro a alguien le sirve.

Deberían ser datos que reflejen lo mejor posible **“lo que nos encontraremos en la realidad”**. Los resultados sobre este conjunto deberían reflejar **qué tan bien funcionará** nuestro sistema en la práctica.



También llamado

- **“test set”,**
- **“eval set”,**
- **“validation set”**
- **“hold-out set”,**
- **“held-out set”,**

Dependiendo de la bibliografía. Es esta materia:

Datos de control



Primer paso: Conjunto de Control

Antes de **hacer cualquier otra cosa** separamos un conjunto de datos que guardaremos en una caja fuerte

(datos_control.zip pass=<cumple_de_pablo>).

- **No se usan hasta el final del desarrollo:**
 - Hasta unos días antes de publicar el paper
 - Hasta unos días antes de subir el sistema a producción
- **¡Se usan una vez! (solo para reportar!)**
 - Para evaluar el modelo que se sube a prod.
 - O para armar la tabla final del paper
 - **No para tomar decisiones. No para elegir los mejores.**
- **"No me gusta lo que me dio":**
 - Ajo y agua. Reportarlo igual que seguro a alguien le sirve.



También llamado

- "test set",
- "eval set",
- "validation set"
- "hold-out set",
- "held-out set",

Dependiendo de la bibliografía. Es esta materia:

Datos de desarrollo

Datos de control



El resto: **Datos de Desarrollo.**

Los usaremos para
entrenamiento, validación,
comparación, análisis, etc.

Etapa de desarrollo: Selección de modelos

Dos tareas importantes en la etapa de desarrollo:

- T_{D1} : ¿Cómo **seleccionar** el mejor modelo entre muchas opciones?
- T_{D2} : ¿Cómo **evaluar** cada uno de manera correcta?

Configuraciones: (ternas <algoritmo, hiperparámetros, elección de atributos>)
(próxima slide definimos hiperparámetros)

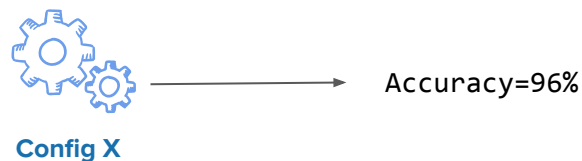
T_{D1} : **Selección de modelos** (Model Selection):

Evaluar distintas **configuraciones** para poder seleccionar la mejor.



T_{D2} : **Evaluación de una configuración dada.**

Asignar un valor a una configuración.



¡Warning!

Sobrecarga de la palabra “**modelo**”. En estos casos se utiliza para referirse a la **configuración** y no la **hipótesis** resultante.

Podríamos llamar a la técnica “Configuration Selection”

T_{D1} : Selección de modelos

Buscando la mejor configuración

Buscando la mejor configuración

Terminología: **Hiperparámetros** vs. **Parámetros**

Hiperparámetro: Se refiere valores que se especifican manualmente en el algoritmo de aprendizaje antes de ejecutarlo.

Parámetro: Se refiere a valores internos del modelo resultante que se aprenden (ajustan) a partir de ejecutar el algoritmo sobre un dataset.

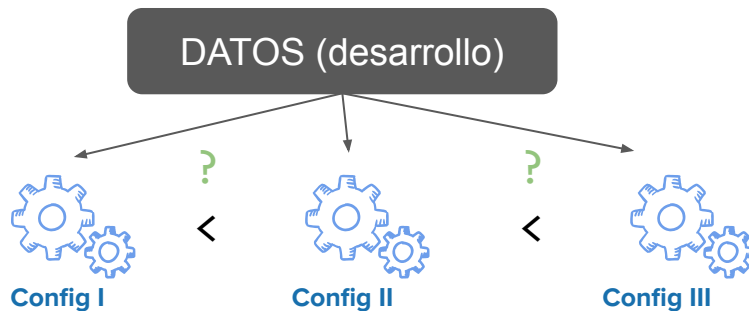
Representan las reglas aprendidas.

----- Por ejemplo en árboles -----

Hiperparámetros: Criterio de elección de atributos en cada nodo (Gini Gain, Information Gain), cantidad de hijos (árboles binarios vs. n-arios), criterio de parada (ej: max_depth), estrategia de poda, etc.

Parámetros: las triplas <nodo_id, atributo, corte> obtenidos. ("Lo que se guarda si hago un **modelo.zip**")

(ver https://scikit-learn.org/stable/model_persistence.html)



¿Cómo buscar la mejor configuración?

Para poder elegir un modelo final, es natural explorar distintas combinaciones de **hiperparámetros**, distintos **algoritmos** de aprendizaje y distintas **formas de extraer o procesar atributos**.

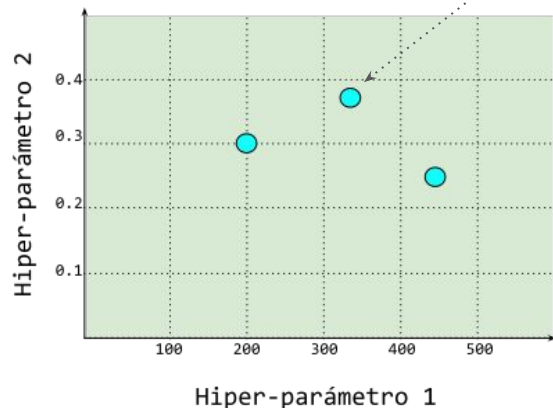
Buscando la mejor configuración

Métodos de exploración de combinaciones: Grid, Random

Conf 83 ($\text{hiper}_1=350$, $\text{hiper}_2=0.4$)
Accuracy: 0.83 (ya veremos cómo calcular esto)

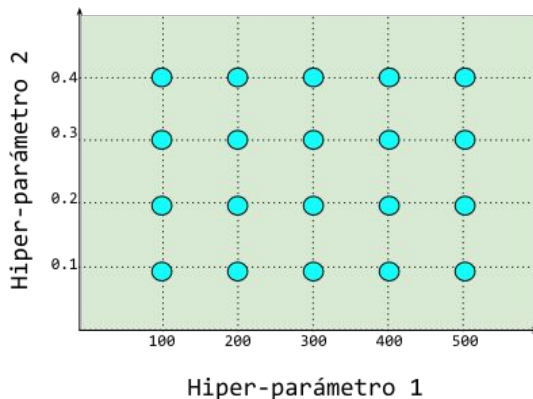
Manual Search

Setear a mano hiper-parámetros que pensamos que van a funcionar bien.



Grid Search

Plantear opciones y explorar todas las combinaciones.



Ejemplo:

- **5** hiperparámetros.
- **10** valores diferentes para cada híper.
- **100,000** (10^5) evaluaciones.
- Entrenamiento del modelo, promedio de **10 minutos**
- La optimización de hiperparámetros nos llevaría casi **2 años**.

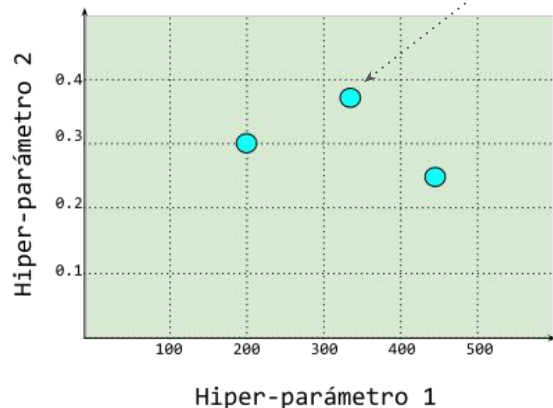
Buscando la mejor configuración

Métodos de exploración de combinaciones: Grid, Random

Conf 83 ($\text{hiper}_1=350$, $\text{hiper}_2=0.4$)
Accuracy: 0.83 (ya veremos cómo calcular esto)

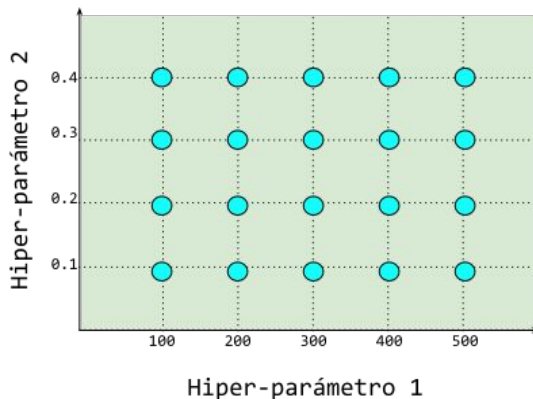
Manual Search

Setear a mano hiper-parámetros que pensamos que van a funcionar bien.



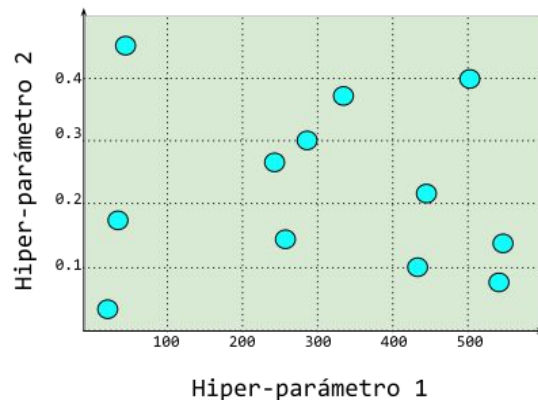
Grid Search

Plantear opciones y explorar todas las combinaciones.



Random Search

Se plantean distintas opciones y se exploran combinaciones al azar.



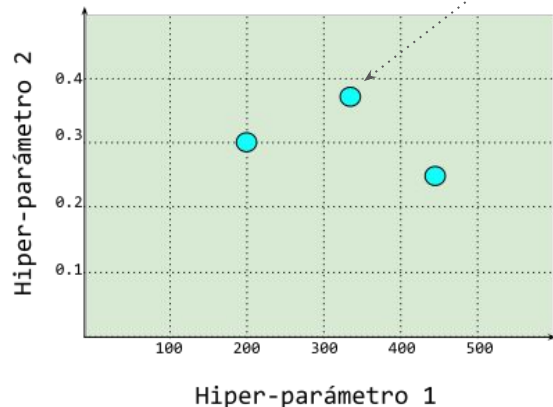
Buscando la mejor configuración

Métodos de exploración de combinaciones: Grid, Random

Conf 83 ($\text{hiper}_1=350, \text{hiper}_2=0.4$)
Accuracy: 0.83 (ya veremos cómo calcular esto)

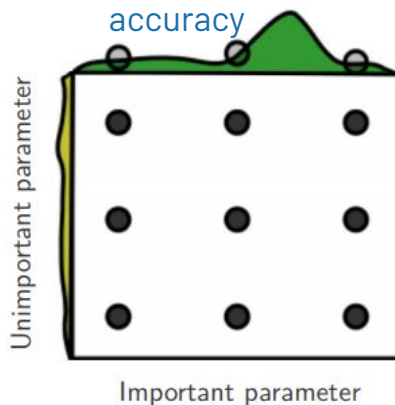
Manual Search

Setear a mano hiper-parámetros que pensamos que van a funcionar bien.



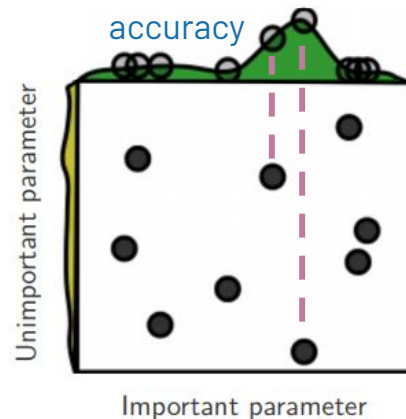
Grid Search

Plantear opciones y explorar todas las combinaciones.



Random Search

Se plantean distintas opciones y se exploran combinaciones al azar.



Buscando la mejor configuración

Métodos de exploración de combinaciones: Random

Especificando distribuciones

En random search, se puede especificar de qué manera muestrear al azar los hiperparámetros.

Ejemplo scikit-learn (SVM)

Parameter name	Distribution	Values
Step size	Log-uniform	$x \in [0.01, 0.5]$
Batch size	Log-uniform integer	$x \in [16, 512]$
Activation function	Categorical	$x \in \{\text{Relu}, \text{PRelu}, \text{Elu}, \text{Sigmoid}, \text{Tanh}\}$
Number of hidden layers	Categorical	$x \in \{1, 2\}$
Number of units in the first layer	Uniform integer	$x \in [50, 1000]$
Number of units in the second layer (In case if it defined)	Uniform integer	$x \in [50, 1000]$
Dropout layer	Uniform	$x \in [0, 0.5]$

```
{ 'C': scipy.stats.expon(scale=100), 'kernel': ['rbf'], 'class_weight': ['balanced', None] }
```

(ver https://scikit-learn.org/stable/modules/grid_search.html#grid-search)

Buscando la mejor configuración

un problema de optimización

```
SMBO( $f, M_0, T, S$ )  
1    $\mathcal{H} \leftarrow \emptyset$ ,  
2   For  $t \leftarrow 1$  to  $T$ ,  
3        $x^* \leftarrow \operatorname{argmin}_x S(x, M_{t-1})$ ,  
4       Evaluate  $f(x^*)$ ,  $\triangleright$  Expensive step  
5        $\mathcal{H} \leftarrow \mathcal{H} \cup (x^*, f(x^*))$ ,  
6       Fit a new model  $M_t$  to  $\mathcal{H}$ .  
7   return  $\mathcal{H}$ 
```

Figure 1: The pseudo-code of generic Sequential Model-Based Optimization.

Buscando la mejor configuración

Métodos de exploración de combinaciones: Optimización Bayesiana



Journal of Global Optimization **13**: 455–492, 1998.

© 1998 Kluwer Academic Publishers. Printed in the Netherlands.

Efficient Global Optimization of Expensive Black-Box Functions

DONALD R. JONES¹, MATTHIAS SCHONLAU^{2,★} and WILLIAM J.
WELCH^{3,★★}

usando la información de lo conocido hasta ahora

Buscando la mejor configuración

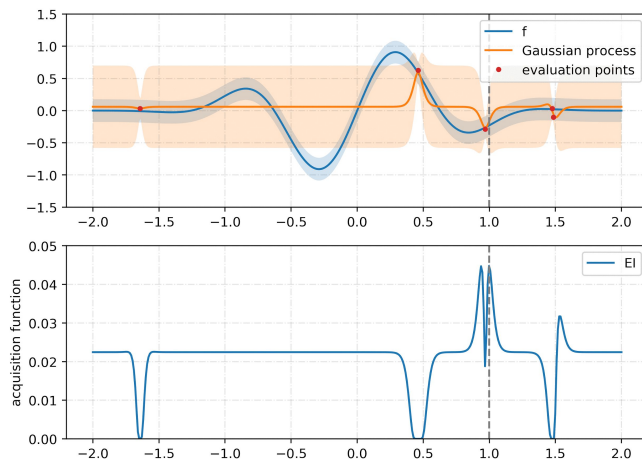
Métodos de exploración de combinaciones: Optimización Bayesiana

Los métodos anteriores **no toman en cuenta** la información provista por **evaluaciones anteriores** para decidir cuál combinación probar a continuación.

Nos interesaría construir un modelo $P(s | h)$ en donde s es un puntaje de error (mientras más chico mejor), y h un valor para el hiperparámetro. Estimar $P(s | h)$, basado en observaciones anteriores, permite buscar en áreas prometedoras.

Métodos de esta pinta (aka **"Secuencial Model Based Optimization"**)

- Gaussian Process with Expected Improvement
- Random Forests (lo veremos más adelante)
- **Tree-structured Parzen Estimators (TPE)**
(a continuación)

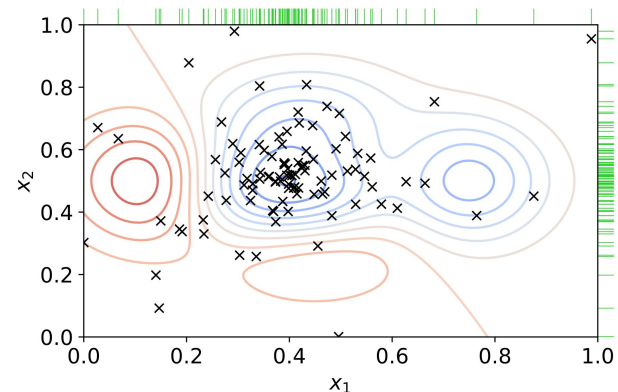
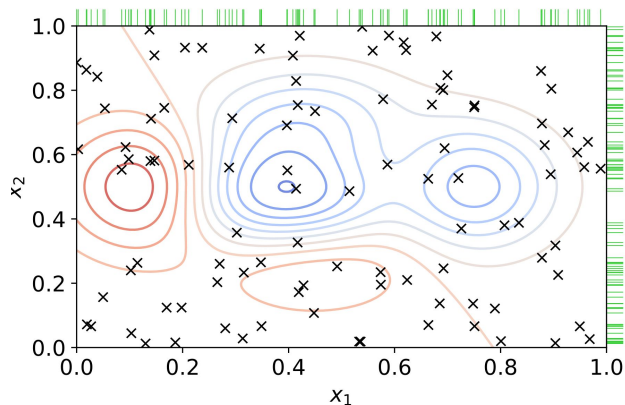
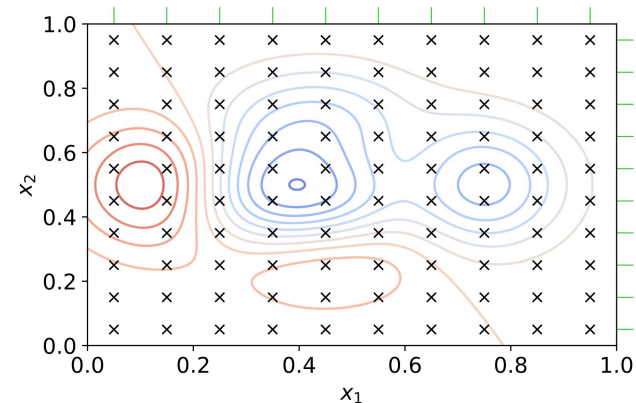


Fuente:

<https://techblog.criteo.com/hyper-parameter-optimization-algorithms-2fe447525903>

Buscando la mejor configuración

Grid vs Random vs Bayes



Buscando la mejor configuración

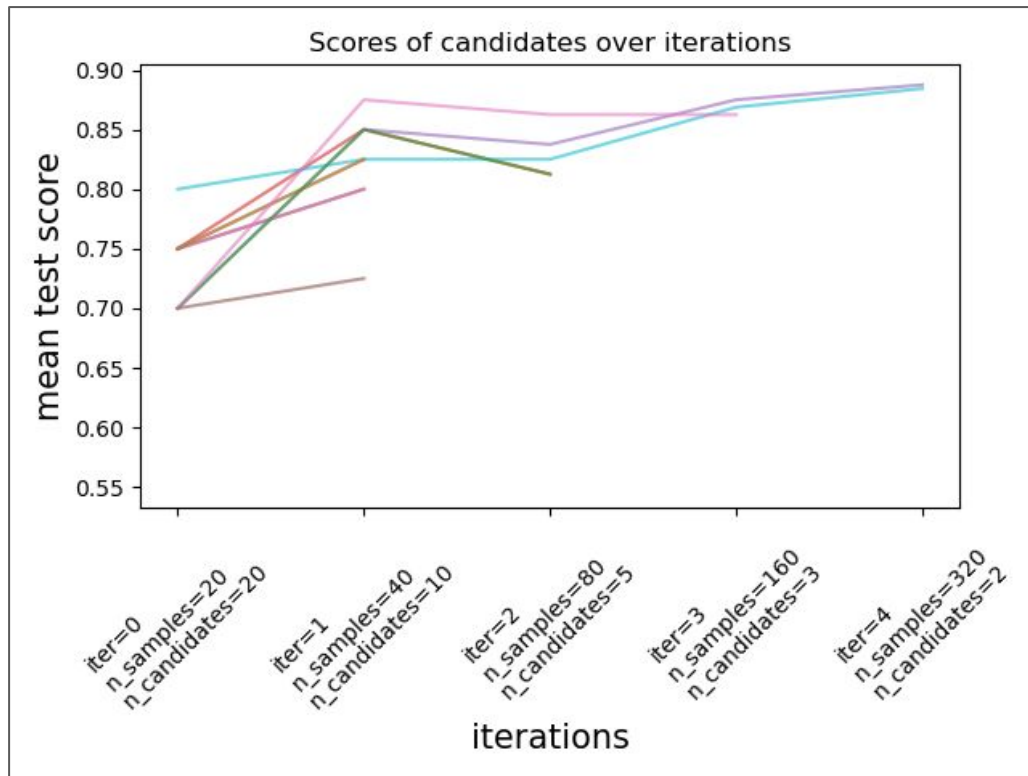
Métodos de exploración de combinaciones: Halving ("reducir a la mitad")

Una alternativa a **grid/randomized search** es:

- 1) Darle pocos "recursos" a las primeras iteraciones (entrenar muchos modelos con pocos datos, con pocas iteraciones)
 - 2) Seleccionar los más prometedores
 - 3) Volver a probar con cada vez más recursos.
- "Halving" Random Search
 - "Halving" Grid Search
 - Hyperband

Ver:

https://scikit-learn.org/stable/auto_examples/model_selection/plot_successive_halving_heatmap.html#sphx-glr-auto-examples-model-selection-plot-successive-halving-heatmap-py



Population Based Training of Neural Networks

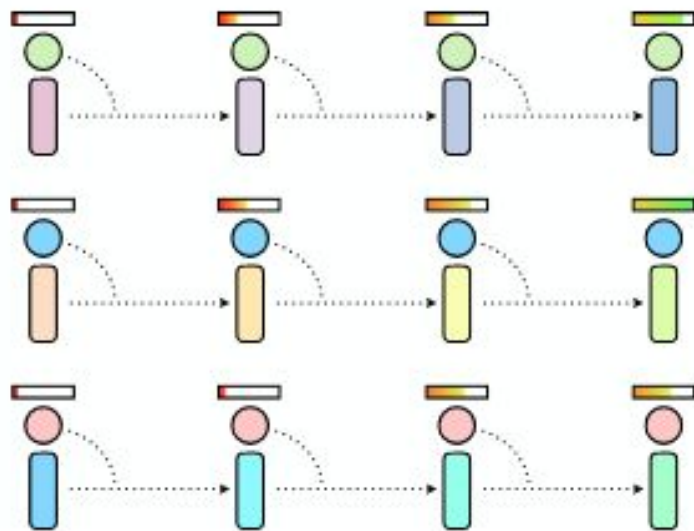
Max Jaderberg Valentin Dalibard Simon Osindero Wojciech M. Czarnecki

Jeff Donahue Ali Razavi Oriol Vinyals Tim Green Iain Dunning

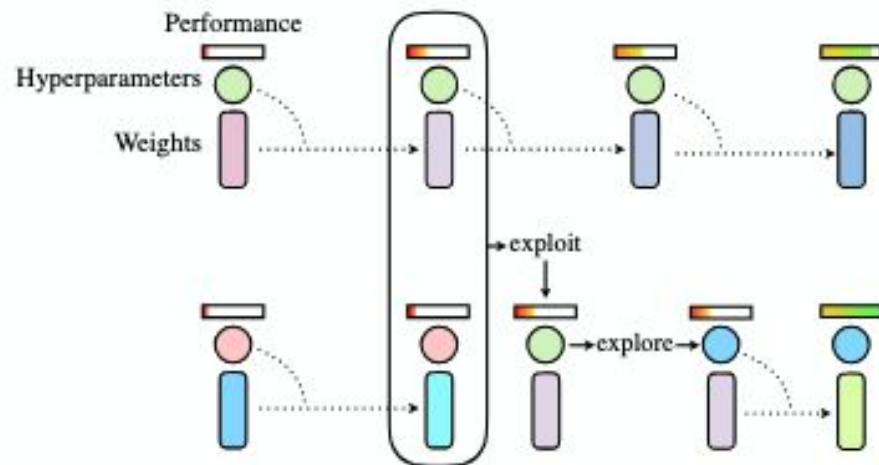
Karen Simonyan Chrisantha Fernando Koray Kavukcuoglu

DeepMind, London, UK

(b) Parallel Random/Grid Search

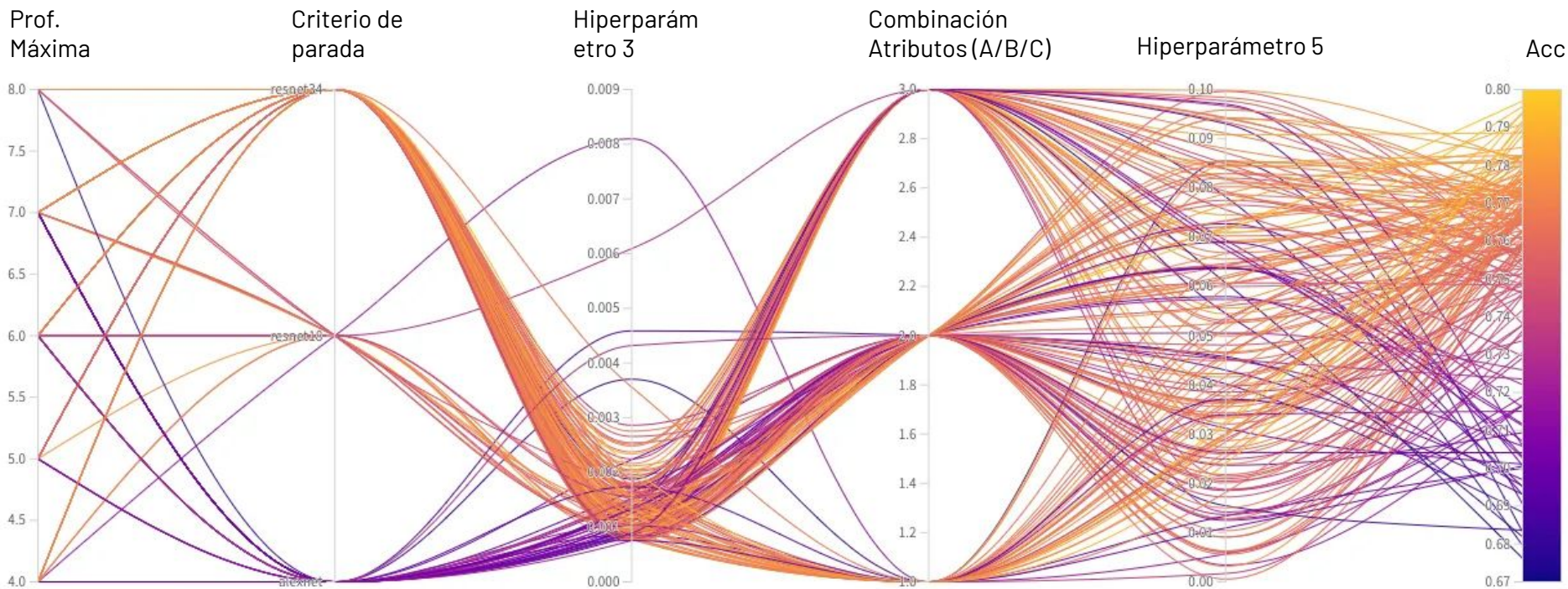


(c) Population Based Training



Buscando la mejor configuración

Ejemplo de visualización de resultados



T_{D2} : Evaluación de una configuración dada

Conf 83 ($\text{hiper}_1=350$, $\text{hiper}_2=0.4$)
Accuracy: 0.83 ??

La expresión “**test**” es ambigua.
En esta materia evitaremos usarla.

Estimación de performance

Validación Cruzada

¿Cómo estimar la performance de una configuración?

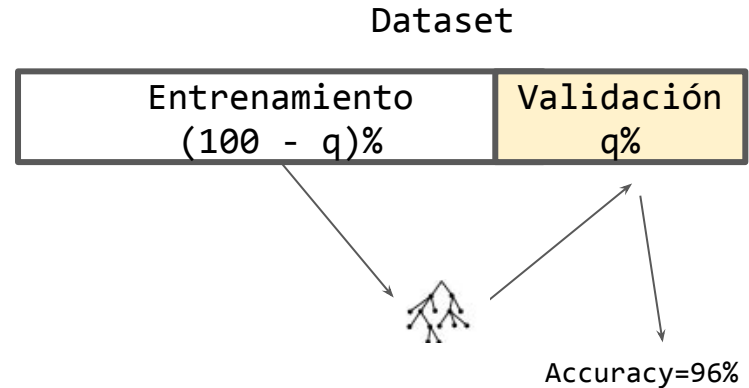
Medir accuracy sobre datos de entrenamiento → mala idea. ¿Por qué?

Surge la necesidad de separar un $q\%$ de datos, para validar los modelos: datos de validación.

Los datos se deben separar al azar^(*) para evitar cualquier orden o estructura subyacente en los datos.

^(*) **Esto no siempre es así.** Veremos en un par de slides

Validación Cruzada (cross validation)^(**)



^(**) Algunos autores llaman “Cross Validation” a lo que veremos en la diapo siguiente y esto lo nombran distinto (train/test split) por ejemplo

Estimación de performance

Validación Cruzada

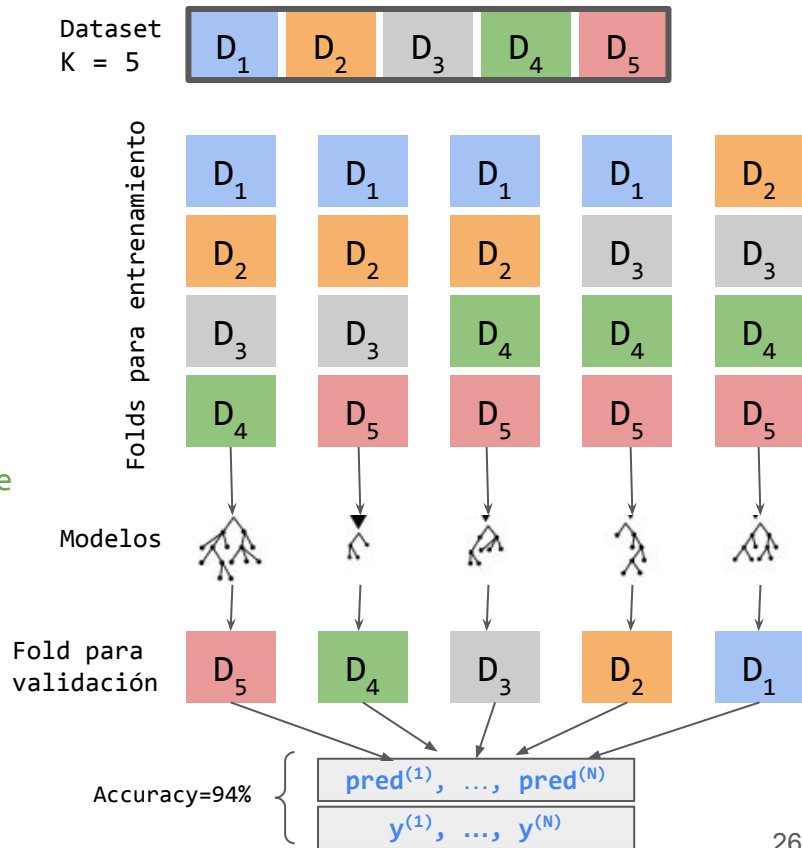
- ¿Qué puede pasar si tenemos mala suerte al separar los datos para entrenamiento/validación? Estamos midiendo sólo sobre el **q%** (en general 5%, 10% o 20%) de nuestros datos. ¿El resultado será confiable? La estimación de performance del modelo podría no ser realista.
- Estos problemas surgen **especialmente** cuando tenemos **pocos datos**, si no, la validación cruzada suele alcanzar.
- Con pocos datos, sería más útil poder probar nuestro algoritmo de aprendizaje con todos nuestros datos. Surge la idea de Validación Cruzada de "k" iteraciones:
"K-Fold Cross Validation"

Estimación de performance

Validación cruzada de K-Folds con métrica "global"

Dada una configuración **c** (un algoritmo de aprendizaje con ciertos hiperparámetros ya establecidos) y el dataset **D**:

1. Separamos **D** en **K subconjuntos** a los que llamamos **folds**:
 $D_1, D_2, D_3, \dots, D_K$.
2. Construimos **K modelos** que serán entrenados utilizando todos los datos (salvo los del k-ésimo conjunto). Es decir, $f_{c, D-D_k} = c(D-D_k)$.
3. Para cada $x^{(i)} \in D$:
 - $\text{pred}^{(i)} = f_{c, D-D_k}(x^{(i)})$
predecimos utilizando el modelo que no vio ese dato en entrenamiento.
 - $\text{predicciones}[i] = \text{pred}^{(i)}$
juntamos las predicciones como si vinieran todas del mismo modelo
4. Computamos alguna métrica del error sobre el conjunto entero $\text{predicciones}[]$ vs $y[]$. Llamaremos a este cómputo "cálculo GLOBAL de la métrica" (slide siguiente, más sobre esto)



Detalles importantes

¿Métrica “global”? ¿Por qué no promediar los resultados de los distintos folds:

- Calcular la métrica para cada fold de validación por separado, luego **promediar** los resultados.
- **No lo recomendamos en general**, salvo que se tengan folds muy grandes y estén muy seguros...
- Pero si los folds son grandes... ¿seguro que queremos k-fold-cross-val? (en gral. no)

Algunos problemas:

- x ¿Si no confiábamos en la robustez de los resultados en un solo fold, por qué confiaríamos en estadísticas (ej promedios) a partir de estos?
- x Promediar algunas métricas (como Accuracy) tiene sentido. Otras como F1 o AUC-ROC, no (más sobre esto en clase métricas).

Perfecto, tengo **K** modelos con k medidas...

¿Cuál uso de ahora en más?

¿Era la idea buscar un modelo en este punto?

(a veces el modelo final lo entrena el usuario)

Opciones (en caso que sí quiera “el modelo final”) (todas con pros y contras):

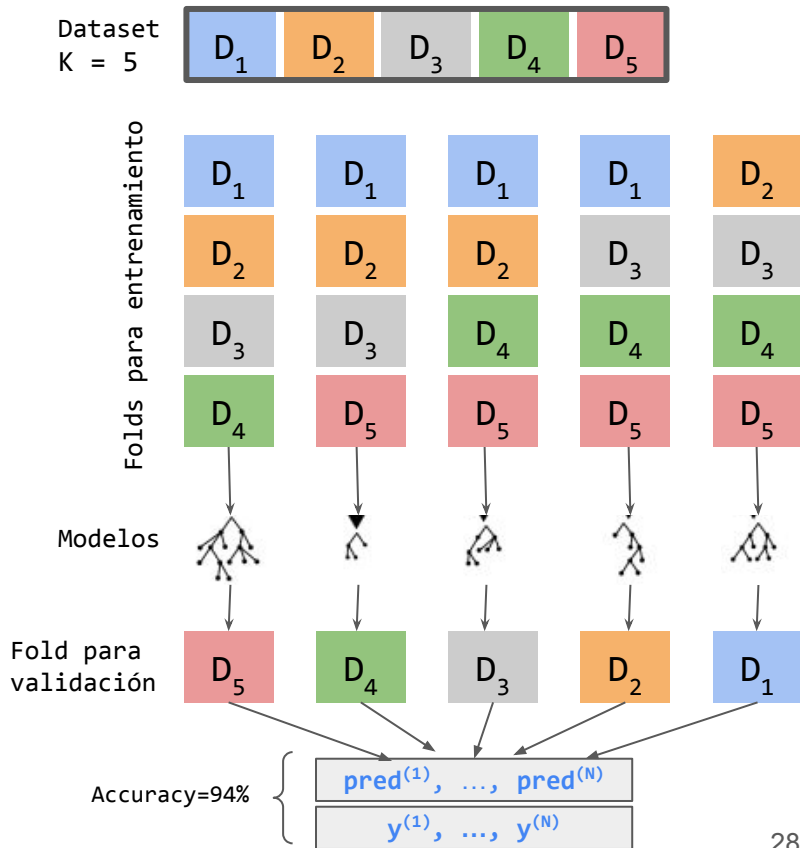
- La más simple... Usar el “mejor” (viendo los resultados por fold)... Ojo.. ¿qué implicancias tiene haber sido el mejor?
- ¿Re-entrenar utilizando todos los datos? La más común, no siempre posible o deseable. ¿Cómo sé que no empeoró?
- Hacer una votación (“Ensamble” de modelos ya entrenados).

Estimación de performance

Validación cruzada de K-Folds

¿Qué sucede si $K = N$? (esto se llama **leave-one-out cross validation**)

- ¿Costo computacional?
- ¿Qué tan similares / distintos son los K modelos en ese caso?
- Si los datos no son realmente independientes (hay un par de instancias similares) una configuración que sobreajusta podría obtener un valor muy alto muy fácilmente. ¿Cómo?



Estimación de performance

Variaciones de K-Fold Cross Validation

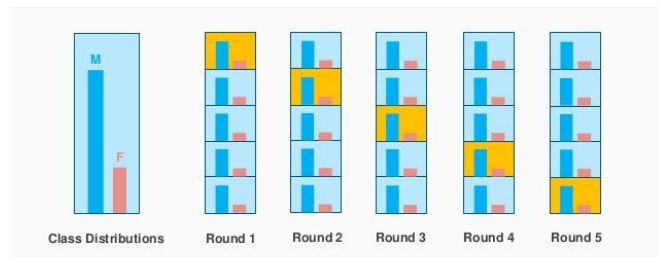
¿Es buena idea hacer **K folds al azar**?

- ¿Las clases están desbalanceadas?
- ¿Orden temporal de los datos?
- ¿Orden espacial? (datos regionales)

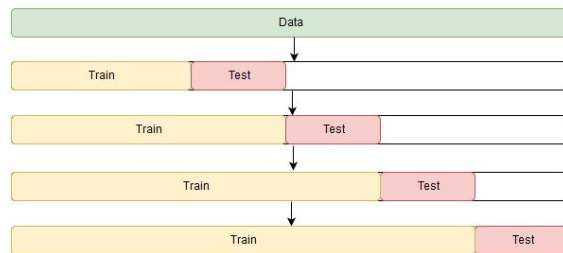
Pensamiento clave **¿Qué queremos aprender?**

Preguntarse entonces: ¿Tengo la misma cantidad de información que tendré en test (o en la realidad)?

Stratified K-fold cross validation.



Temporal series K-fold cross validation.



Problema típico

Ejemplo: Si un médico sospecha que alguien tiene cáncer de pulmón, te manda a la máquina del hospital avanzado "A". Si piensa que no lo tenés, pero para descartar, te mandan al hospital "B" (en donde las fotos salen un poco más claritas) ... Entreno mi modelo, predice al 99%.

Me doy cuenta que usa fuertemente la claridad en la imagen.

Pregunta: ¿Quiero mejorar mi predicción a costa de esta info?

Respuesta: En general NO, ya que son correlaciones espurias (no generalizará bien).

Clave, separar por "condición"

"Condición": Grupos en la data que introduce correlaciones espurias o facilitan la tarea que no quisiera usar en el futuro.

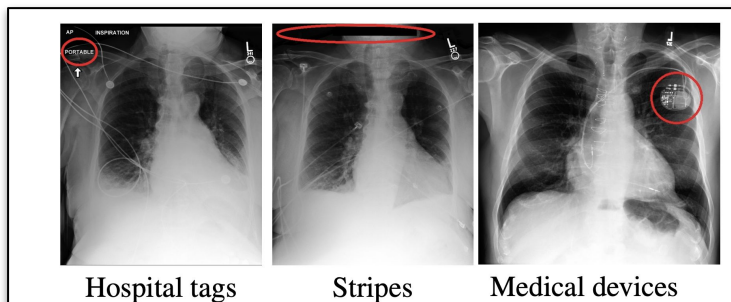


Figure 2: Hospital tags, stripes, and medical devices exemplify several unknown group labels in the MIMIC-CXR dataset, which might spuriously correlate with the ground truth diagnosis results.

YE, Wenqian, et al. **Spurious correlations in machine learning: A survey.** 2024.
<https://arxiv.org/pdf/2402.12715v1>

Problema típico

Para pensar ahora.. ¿Cómo separarían los datos?

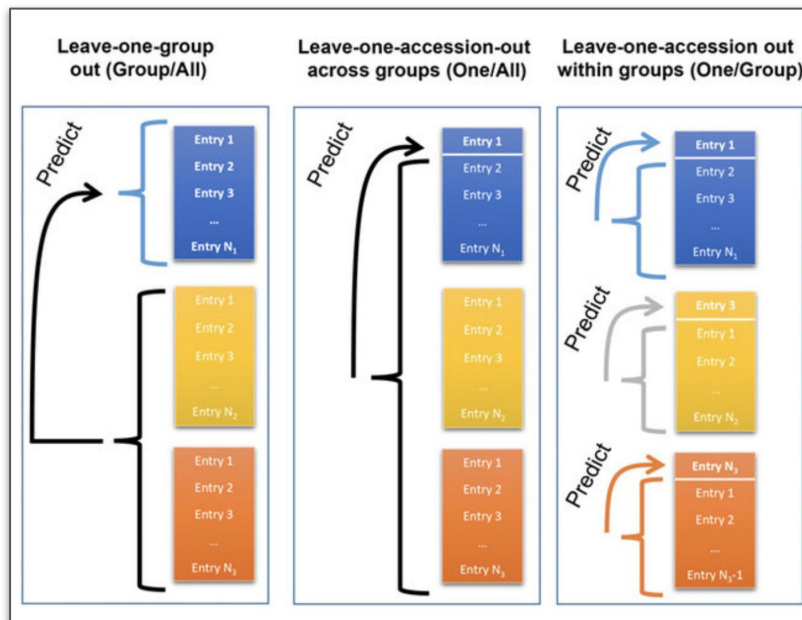
- **Ej₁: Detección de emociones**, tenemos 2000 instancias, pero de 10 hablantes. ¿Cómo hacemos las particiones en folds? ¿Cuál es la pregunta? ¿A qué quiero generalizar?
- **Ej₂: Detección de covid en toces**: Micrófono utilizado. ¿puede un modelo aprender a detectar el micrófono? ¿Separo también los distintos hablantes en los folds?

Ver Ejercicio 4.4 de la guía

Group-k-fold cross validation:

K-fold cross val en donde nos aseguramos que datos de diferentes grupos no se mezclen entre los conjuntos de entrenamiento y prueba.

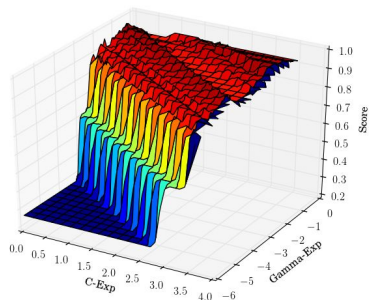
Generalmente los grupos se encuentran en la **metadata**.



Fuente: [Prospects of genomic prediction in the USDA Soybean Germplasm Collection: Historical data creates robust models for enhancing selection of accessions]

Volviendo a la selección del mejor modelo

Una vez probadas distintas combinaciones de estructura del modelo e hiperparámetros obtenemos un **puntaje por cada combinación**



(Utilizando cross validation)

Una vez seleccionados los mejores hiperparámetros, **podemos entrenar un modelo utilizando todos nuestros datos.**

Escenario frecuente:

Construimos nuestro modelo con la combinación de **atributos + algoritmos + hiperparámetros** con mejor desempeño.

Lo ponemos a funcionar con datos nuevos, y los resultados son **peores**.

¿Qué falló?

A otro nivel, repetimos el mismo error de antes. Evaluamos un modelo sobre los **mismos datos** que usamos para **construirlo (esta vez no “para entrenarlo”)**.

- Entonces, **sobreestimamos** la performance de nuestro modelo.
- ¿Solución?

¿Por qué daría mal?

Ya vimos que nuestros modelos pueden sobreajustar en el conjunto de entrenamiento.

Sobreajuste en el conjunto de prueba: Otra noción de sobreajuste es la brecha entre el rendimiento en el conjunto de prueba y el rendimiento en la distribución subyacente de los datos.

Al adaptar las elecciones de diseño del modelo al conjunto de prueba, la preocupación es que **ajustemos implícitamente los hiperparámetros** al conjunto de prueba. La precisión en el conjunto de prueba, entonces, pierde su validez como una medida precisa del rendimiento en datos verdaderamente no vistos.

Opinión personal:

Una de las causas es:

regression to the mean

(*regression to the mediocrity*)

<https://www.youtube.com/watch?v=1tSqSM0yNFE>

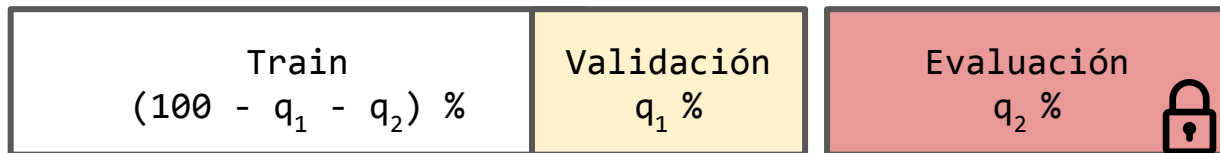
(Veritasium)

Evaluación del modelo final

Resumen Desarrollo - Evaluación

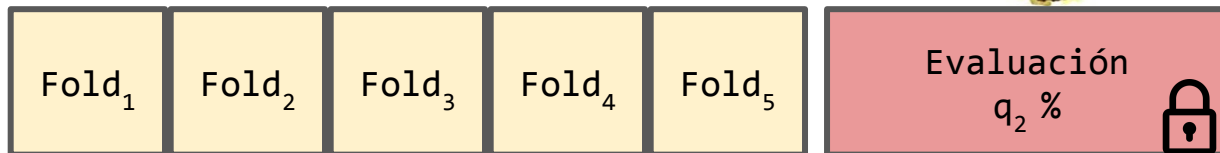
- Lo antes posible separamos un **conjunto para evaluación**.
- Serán datos que no se toquen **hasta el final**.
- Todas las pruebas y ajustes se hacen sobre el conjunto para **Desarrollo**.
- Cuando **termina el desarrollo**, se **evalúa** sobre los datos de evaluación separados.
- La estimación de performance será la más realista.
- Ese valor es el que se reporta.
iNo volver atrás!

Caso general



Desarrollo

Caso pocos datos
(K-fold cross val)



Desarrollo



Desarrollo - Evaluación

importante: Los eval sets deberían ser actualizados constantemente!

Ejemplo: <https://arxiv.org/abs/1806.00451>

Recrearon un dataset equivalente al original y notaron que los modelos bajaron su performance de 4% to 10%

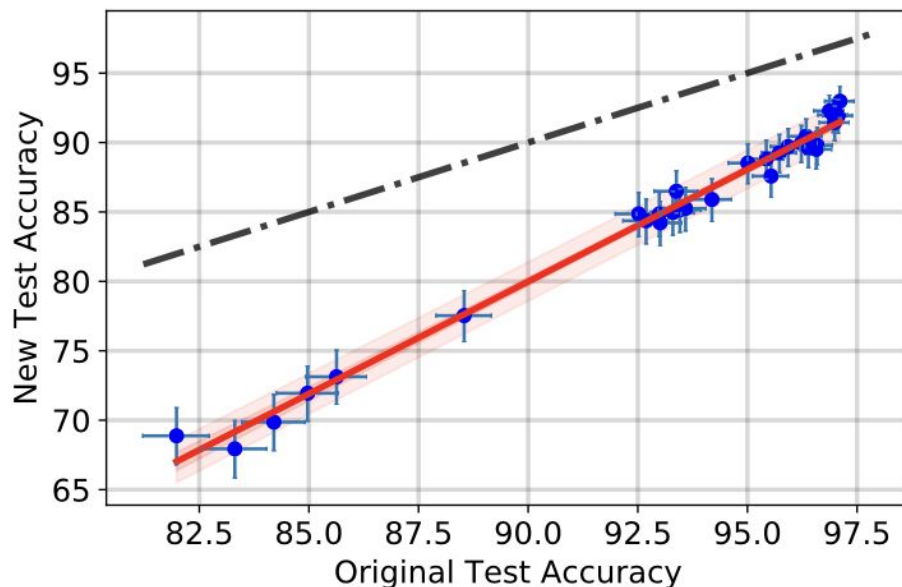
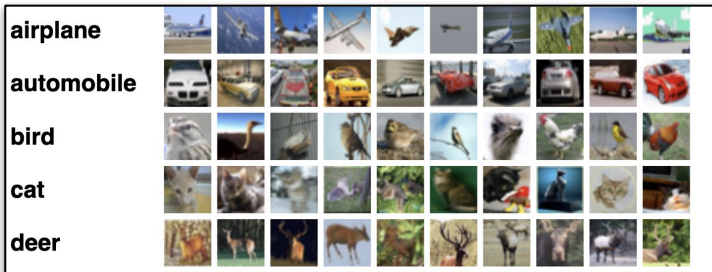
Do CIFAR-10 Classifiers Generalize to CIFAR-10?

Benjamin Recht
UC Berkeley

Rebecca Roelofs
UC Berkeley

Ludwig Schmidt
MIT

Vaishal Shankar
UC Berkeley



	Original Accuracy	New Accuracy	Gap	Δ Rank
shake_shake_64d_cutout [3, 4]	97.1 [96.8, 97.4]	93.0 [91.8, 94.0]	4.1	0
shake_shake_96d [4]	97.1 [96.7, 97.4]	91.9 [90.7, 93.1]	5.1	-2
shake_shake_64d [4]	97.0 [96.6, 97.3]	91.4 [90.1, 92.6]	5.6	-2
wide_resnet_28_10_cutout [3, 22]	97.0 [96.6, 97.3]	92.0 [90.7, 93.1]	5	+1
shake_drop [21]	96.9 [96.5, 97.2]	92.3 [91.0, 93.4]	4.6	+3
shake_shake_32d [4]	96.6 [96.2, 96.9]	89.8 [88.4, 91.1]	6.8	-2
darc [11]	96.6 [96.2, 96.9]	89.5 [88.1, 90.8]	7.1	-4
resnext_29_4x64d [20]	96.4 [96.0, 96.7]	89.6 [88.2, 90.9]	6.8	-2

TAREA

- Leer el **capítulo 5 “Model Evaluation and Improvement”** (hasta Evaluation Metrics and Scoring, **sin incluir**) de Müller, A. C., & Guido, S. (2016). Introduction to machine learning with Python: a guide for data scientists.
- Guía + Cuestionario

(Opcional, recomendadísimo)

Curso realizado por **Luciana Ferrer**

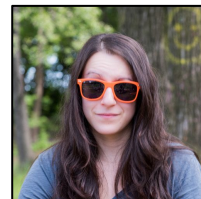
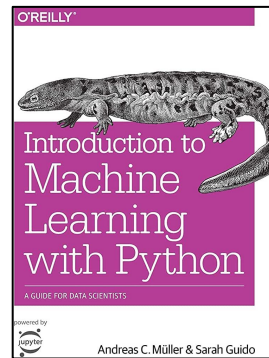
(<https://liaa.dc.uba.ar/es/nosotros/>). Organizado por Vía Libre y FAMAf - Universidad Nacional de Córdoba.

[Evaluación responsable de sistemas de IA](#)

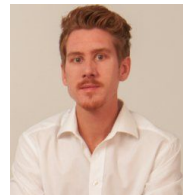
(Opcional, quizás para después de la clase 5) Raschka, S. (2018).

Model evaluation, model selection, and algorithm selection in machine learning. arXiv preprint arXiv:1811.12808.

<https://arxiv.org/abs/1811.12808>



Andreas C. Müller Sarah Guido



Sebastian Raschka